Day 06 Report: Emotion Recognition in Text

BiLSTM with GloVe embeddings vs a fine-tuned DistilBERT on 6 emotions (dair-ai/emotion), Colab T4 GPU

Summary

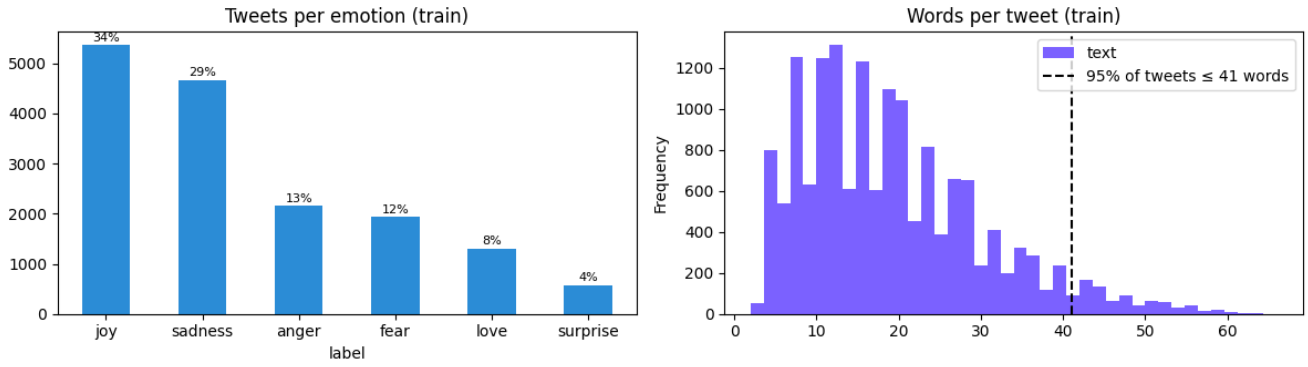
The **BiLSTM with GloVe** and the fine-tuned **DistilBERT** scored almost the same: **92.8% vs 92.9% accuracy** and **0.890 vs 0.888 macro F1** on 2,000 test tweets. The BiLSTM trained in **0.4 minutes** and is about **6 MB**; DistilBERT took 3.0 minutes and is about 268 MB. A classic TF-IDF baseline reached 86.3% and is used in my Day 19 capstone.

1. Goal

Go beyond positive/negative sentiment: classify English tweets into six emotions (sadness, joy, love, anger, fear, surprise), a key skill for empathetic chatbots and customer-support tools.

2. Data

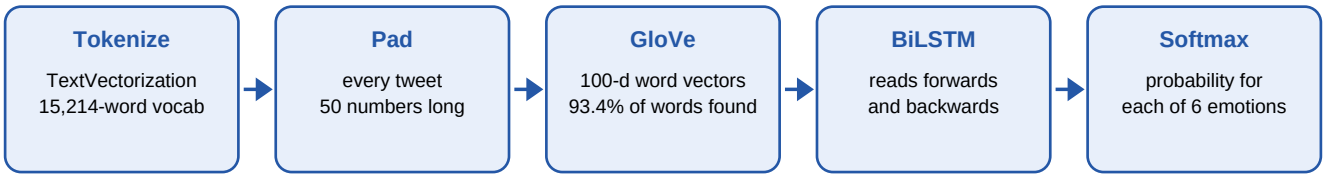
dair-ai/emotion (Hugging Face): 16,000 training, 2,000 validation and 2,000 test tweets, already lower-case. The classes are imbalanced: joy has 5,362 training tweets but surprise only 572. So I used **class weights** and report **macro F1** (every emotion counts equally), not only accuracy.



Tweets per emotion (left) and words per tweet (right); 95% of tweets have 41 words or fewer

3. How it works

Model 1: BiLSTM pipeline

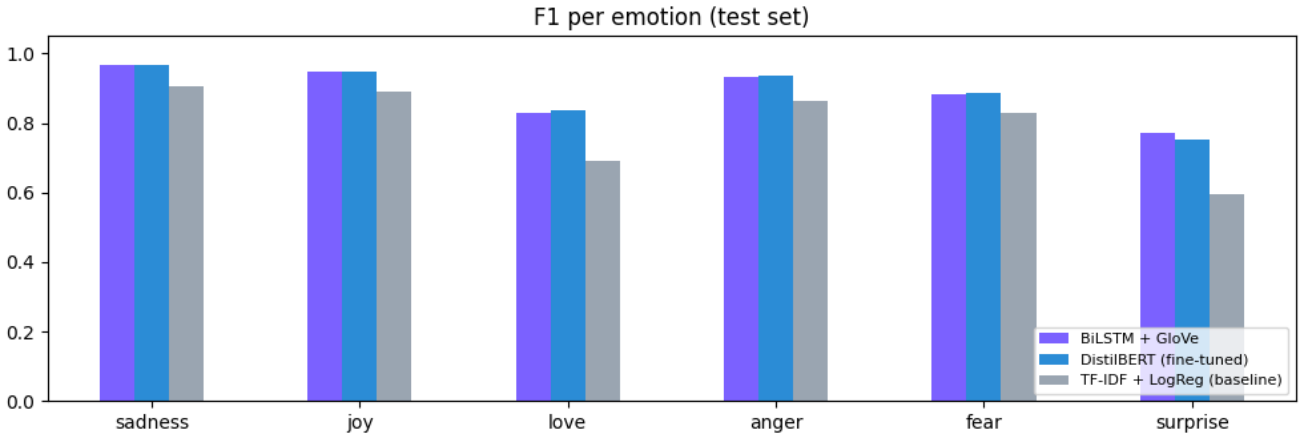


Step	What I did	Why
Tokenization + padding	Keras TextVectorization (20,000 max words), sequences padded to 50	Networks need numbers of equal length
Word embeddings	GloVe glove-wiki-gigaword-100 as the embedding matrix (14,212 of 15,212 words found), fine-tuned	Words with similar meaning start close together: angry -> furious, outraged; scared -> frightened, afraid
BiLSTM	Bidirectional LSTM(64), dropout, Dense(64), softmax; Adam; class weights; early stopping	Reading both directions keeps context such as "not happy"
DistilBERT	distilbert-base-uncased, fine-tuned 2 epochs (lr 5e-5, batch 32, 64 tokens) in PyTorch	A small pre-trained Transformer: each word's meaning depends on the whole sentence
Baseline (extra)	TF-IDF (1-2 word grams) + Logistic Regression	Fast, tiny, CPU-only reference point

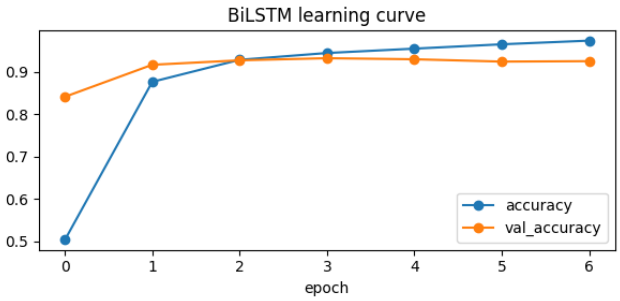
4. Results (test set, 2,000 tweets)

Model	Accuracy	Macro F1	Train time	Size
BiLSTM + GloVe	92.8%	**0.890**	0.4 min	~6 MB
DistilBERT (fine-tuned)	**92.9%**	0.888	3.0 min	~268 MB
TF-IDF + LogReg (baseline)	86.3%	0.796	seconds	~2 MB

F1 per emotion	sadness	joy	love	anger	fear	surprise
BiLSTM + GloVe	0.968	0.949	0.830	0.933	0.884	0.773
DistilBERT	0.967	0.948	0.837	0.937	0.885	0.752
TF-IDF + LogReg	0.906	0.892	0.692	0.862	0.827	0.596

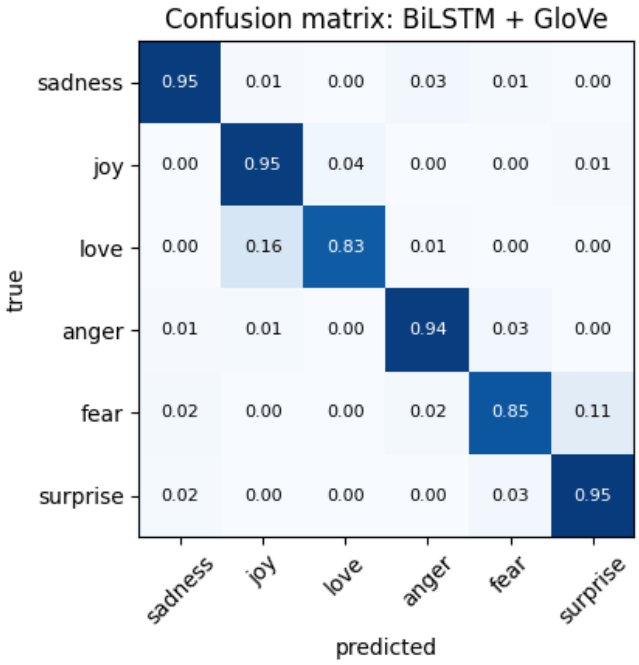


F1 per emotion: the two neural models are almost identical; the baseline is weakest on love and surprise



BiLSTM: validation accuracy peaked at 93.2% (epoch 4); early stopping kept the best weights

5. Error analysis



Confusion matrix of the best model by macro F1 (BiLSTM), each row sums to 1

True emotion	Predicted as	Share	Why it is hard
love	joy	16%	Both are positive; "I feel blessed" can be either
fear	surprise	11%	Both describe a sudden reaction ("shocked", "strange")
joy	love	4%	Words like "passionate" or "caring" appear in both

Surprise is the hardest class (F1 0.75-0.77): it has only 66 test tweets, and the BiLSTM's class weights raise its recall (0.955) at the cost of precision (0.649).

6. Conclusion

- **On this dataset the BiLSTM + GloVe matched DistilBERT** (macro F1 0.890 vs 0.888) while training 7x faster and being about 45x smaller. Short tweets with clear emotion words do not need a Transformer's full context.
- Both neural models beat the TF-IDF baseline by about 9 macro-F1 points, mostly on love and surprise.
- For a real support bot I would use the BiLSTM (fast, small, accurate), or DistilBERT if messages are longer and more subtle.
- My Day 19 capstone uses the TF-IDF baseline from this notebook (emotion_model.joblib): it runs on any CPU without TensorFlow.

Next steps

- Train DistilBERT for 3-4 epochs and tune the learning rate; it may then pull ahead.
- Try a GRU and compare with the BiLSTM.
- Collect more love and surprise examples, or use data augmentation, to fix the weakest classes.